

Generative Specification — The Seven-Property Rubric Scoring Guide

Companion to: *Generative Specification: A Discipline of Derivability for the Stateless Reader*

Scope: the calibration anchors promised in §4.4 of the Compendium — per-property, per-level (0 / 1 / 2) reference criteria, plus each property’s lifecycle and organizational manifestation. **Status:** canonical companion (public). The property definitions are authoritative in Compendium §4.4; this document operationalizes them. **Version:** 1.0 **Date:** June 2026 **Repository:** github.com/jghiringhelli/generative-specification **License:** CC-BY 4.0. Reuse with attribution. **Contact:** jcghiri@gmail.com

Purpose and Scope

Compendium §4.4 defines the seven specification properties and states that the rubric’s reproducibility — that two independent assessors arrive at the same score given the same artifact set — is grounded by **calibration anchors**: per-property reference exemplars at each scoring level (0, 1, 2). It promises that “a score is admissible only when the assessor can name the anchor it is closest to,” but it does not lay the anchors out. This guide fills that gap.

It also makes explicit a dimension the Compendium prose carries but never tabulates, and the dimension the user-facing diagnostic depends on most: **each property’s lifecycle and organizational manifestation** — how the property shows up across the development lifecycle (dev → staging → production → evolution) and toward the three audiences a specification serves beyond the codebase (the outside world / consumers, the company, the team).

This is the precise respect in which the GS rubric differs from SOLID. SOLID is a **static design rubric**: it scores the shape of code at a moment, for a human reader who carries context between sessions. The seven GS properties are a **lifecycle and manifestation rubric**: they score whether a specification keeps producing correct, derivable, recoverable output as the system moves through its lifecycle and as the people around it rotate. A SOLID violation degrades a human’s ergonomics now; a GS violation propagates a derivability failure forward in time — silently, at generation speed, across every session and every successor who inherits the artifact set. The anchors below are therefore phrased against *what is observable over the lifecycle and across the organization*, not only against a snapshot of structure.

The rubric is **0 / 1 / 2 per property, 14 points total** (Compendium §4.4). This guide is faithful to the definitions in §4.4 and to the twenty-nine-pathology catalog (pragmaworks.dev/diagnostico); it introduces no new properties and no new pathologies.

How to score (assessor protocol)

1. For each property, read the **Definition** and the **Lifecycle & organizational manifestation**.
2. Run the **automatable signal** where the harness can produce it; form the **human-judgment signal** by inspection of the artifact set as a stateless reader.
3. Select the **0 / 1 / 2 anchor** the artifact set is closest to. A score is admissible only when you can name the anchor.
4. Where two assessors disagree, they reconcile by naming anchors, not by re-arguing definitions — that is the mechanism the anchors exist to provide.

5. **Executable** is scored conditional on specification availability (§4.4): scored 0–2 when a formal behavioral contract exists to run; scored **N/A** for a goal-directed program with only human acceptance criteria. An N/A on Executable is recorded, not counted as 0; the denominator drops to 12.

A note on independence (§4.4): **Verifiable** establishes that the check infrastructure *exists*; **Executable** establishes that the implementation *passes those checks against a live environment*. They are scored separately and can diverge — a system can be 2 on Verifiable and 0 on Executable. Do not collapse them.

A note on the human ceiling (§4.4): CI can verify six of the seven properties automatically. **Defended is the exception** — whether adversarial challenge has been anticipated and answered requires human review. The Defended anchors below therefore lean harder on the human-judgment signal, and a fully automated `gs_score` should flag Defended as provisional.

The Seven Properties

Throughout, the *artifact set* means the specification surface a stateless reader is given: the architectural constitution (`CLAUDE.md` or equivalent), the functional/technical specification, the use cases, the ADR record, the test and gate corpus, and the status/memory artifacts.

1. Self-describing

1.1 Definition. The system explains its own architecture, decisions, and conventions from its own artifacts; no external knowledge is required. It addresses the *rationale layer* — not just what the structure is, but why it is that way and what rules govern it. Automatable checks (§4.4): (1) presence of an explicit intent statement; (2) presence of a scope boundary statement. A specification lacking either fails regardless of prose quality.

1.2 Lifecycle & organizational manifestation. - **Dev:** a new AI session, given the artifact set alone, can state what the system is and what it is *not* for before writing a line. The first search is deterministic because the convention is written, not inferred. - **Staging:** a reviewer can tell whether a change conforms to stated intent without asking the author what they meant. - **Production:** an on-call responder can derive *why* a component exists and what it is allowed to touch from the artifacts, with no one present to ask. - **Evolution:** a proposed change can be classified as in-intent or out-of-intent by inspection; the spec, not a person, is the arbiter of scope. - **Toward the world:** the public surface (README, API intent, screaming-architecture layout) announces purpose; an external integrator does not need a call to understand what the boundary is for. - **Toward the company:** the system is legible to a stakeholder who never reads code — intent and scope are stated in language-neutral terms. - **Toward the team:** onboarding is reading, not shadowing. A new hire derives the system from artifacts; tribal narration is not on the critical path.

1.3 Scoring anchors.

Level	Anchor
0	No explicit intent statement and/or no scope boundary statement. Architecture lives in people's heads or chat history. A stateless reader cannot state what the system is for, and "what should not change" is undiscoverable. (Closest pathologies: <i>Specification Absence</i> , <i>Implicit Architecture</i> .)
1	Intent and scope are present but partial: the system is <i>documented</i> (describes what was built) rather than <i>specified</i> (derivable for what should be built). Conventions exist but some are unstated; the reader can describe the system but would still need to ask a colleague to act correctly at the edges. (Closest pathology: <i>Specification Debt</i> .)
2	Both automatable checks pass: an explicit intent statement and an explicit scope boundary statement are present, conventions (naming, placement, routing) are stated, and the rationale layer is recoverable. A stateless reader can derive what should and should not change without external knowledge. Reference exemplar (§4.4.1): <pre>pragmaworks CLAUDE.md — a navigation root where every document location announces its domain.</pre>

- **Automatable signal:** presence of an intent statement and a scope-boundary statement in the constitution; sentinel-tree root present and within the bounded line budget; first-search determinism (placement announces purpose).
- **Human-judgment signal:** read the artifact set as a stateless agent who has never seen the project — if you would need to ask something not written down to act correctly, that is the Self-describing gap.

1.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
Specification Absence (02)	0	Write the generative specification: functional spec + architectural constitution + use cases + ADRs; apply the derivability test.
Implicit Architecture (03)	0	Architectural constitution covering identity, standards, constraints/prohibitions, tool sequencing, routing; audited against the five-category grid before first implementation.
Specification Debt (05)	1	Run the derivability test against the artifact set; fill each spot where a stateless reader would have to ask.
Prompt Illiteracy (21)	1	If few-shot examples are needed, the spec has a gap — write the missing constraint, do not improve the prompt.
UI Pattern Drift (26)	1	UX pattern documents (error/loading/empty/modal/tables) as first-class spec artifacts, referenced before any UI generation.

2. Bounded

2.1 Definition. Every unit of work has explicit scope and seams; functions do one thing; modules own one concern. The line limit carries a mechanical justification: AI read operations are capped at a fixed line budget — a file that exceeds it is silently truncated, and the agent edits against an incomplete view; a specification artifact that exceeds the read budget is, from the executor's perspective, equivalent to one that does not exist. At scale, Bounded applies to the session context itself via the **sentinel navigational tree**: a lossless hierarchy where the root is always loaded and the agent descends only the relevant path. A well-formed tree collectively contains five categories — architectural identity, standards, constraints/prohibitions, tool sequencing, routing — of which **tool sequencing is the most commonly absent and most consequential**.

2.2 Lifecycle & organizational manifestation. - **Dev:** the agent loads only the slice the task needs; no spec file is truncated; the change surface for any task is a bounded, declared seam. - **Staging:** review

scope equals the seam — a reviewer reads the boundary declaration, not the whole graph, to judge a change. - **Production:** a fault is localizable to one bounded concern; blast radius is the seam, not the system. - **Evolution:** adding a node does not require reading every node; the tree stays lossless when joined but bounded per session — token cost does not grow with system age. - **Toward the world:** module/service boundaries are the integration contract; a consumer depends on a declared seam, not an internal. - **Toward the company:** cost of any change is estimable because scope is explicit; “we’d have to grep” is replaced by “the change surface is this seam.” - **Toward the team:** parallel work without collision — two people work two seams without coordinating internals, because the seams are declared.

2.3 Scoring anchors.

Level	Anchor
0	Specification or constitution files exceed the read budget (silently truncated) and/or modules have no declared seams. The bottom of the constitution is invisible to the agent; rules at the bottom are never read. Units do many things; scope of any change is “we’d have to grep.” (Closest pathologies: <i>Context Window Blindness</i> , <i>Context Fragmentation</i> .)
1	Files are within budget and modules have nominal boundaries, but the sentinel tree is incomplete — typically the routing or tool sequencing category is missing (tools are listed but not ordered: “these tools exist,” not “use X before Y when C”). The agent must infer scope at the edges; isolation holds within seams but not reliably across them.
2	All spec artifacts are within the read budget; modules own one concern with explicit seams; the sentinel tree is lossless and present, with all five categories — including explicit tool sequencing. The change surface for any unit is its boundary declaration. Reference exemplar: the AX adversarial series, where Bounded violations (e.g. direct DB access bypassing the repository layer, module-boundary and duplication breaches) are machine-counted by external structural analysis across all conditions.
Note	Complete independent isolation of a unit also requires Composable (§6); Bounded supplies the predictably-scoped context that lets isolation hold across seams.

- **Automatable signal:** line count per spec artifact against the read budget (≈ 300 lines / root index ≤ 200); machine-count of boundary violations (e.g. layer-bypass) per submission; presence of the five tree categories; presence of explicit tool-sequencing clauses.
- **Human-judgment signal:** does each unit do one thing? Can the agent locate the relevant boundary without traversing the full artifact set?

2.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
Context Window Blindness (08)	0	Keep the constitution $\leq \sim 300$ lines as a skeleton with pointers; sentinel navigational tree with bounded children.
Context Fragmentation (07)	0	Consolidate into the artifact set; bounded context-loading strategy (max ~ 5 artifacts at session start).
Component Reinvention (28)	1	Atomic-design structure as a spec artifact; the library is the mandatory first source — bounds the change surface.

3. Verifiable

3.1 Definition. The correctness of any output can be checked without human judgment. Types, tests, lint, coverage and cyclomatic-complexity gates, mutation testing, dependency-vulnerability scans, and

schema contracts form a continuous verification layer. The structural point: agents report file-write success when bytes reach disk, not when code compiles — without an explicit verification layer the agent’s “done” and the developer’s “done” refer to different states. Verifiable closes this structurally: **the verification layer is not optional post-work; it is the definition of completion.** Verification is automatic, fast, and blocking. Tests carry an adversarial role: written against interfaces to detect contract violations, not to confirm the current implementation. Verifiable establishes that the check *infrastructure exists*; whether the implementation passes those checks in a real environment is Executable (§7), scored separately.

3.2 Lifecycle & organizational manifestation. - **Dev:** “done” is defined by passing checks, not by code written; the agent cannot mark a task complete that the verification layer rejects. - **Staging:** the gate, not a reviewer’s attention, certifies correctness; CI is the certifying authority. - **Production:** regressions are caught by the suite before ship, not by users after; the suite detects contract violations, including those that preserve internal structure. - **Evolution:** a correct refactor passes (tests verify interfaces, not internal state); the verification layer survives change instead of breaking on it. - **Toward the world:** schema/contract checks guarantee the published interface behaves as specified. - **Toward the company:** quality is a measured property (mutation score, coverage on changed code), not a claim — auditable to compliance and procurement. - **Toward the team:** review is reallocated from correctness-checking (automated) to judgment; the team does not rubber-stamp because the gate already decided correctness.

3.3 Scoring anchors.

Level	Anchor
0	No blocking verification layer, or tests that assert nothing meaningful: high line coverage with near-zero mutation score — <i>Test Theater</i> . The suite never fails; correctness is asserted by humans or not at all. Completion = bytes on disk. (Closest pathologies: <i>Test Theater</i> , <i>No Code Review Culture</i> .)
1	A verification layer exists and blocks (types, lint, unit tests in CI), but it is not adversarial: tests are shaped to the implementation rather than the contract; mutation score is unmeasured or below gate; refactors break tests that verify internal state. Correctness is partially machine-checked.
2	The verification layer is automatic, fast, and blocking, and it is adversarial: tests target interfaces and detect contract violations; mutation testing gates new/changed code (e.g. MSI $\geq 65\%$ overall, $\geq 70\%$ on changed code); NFRs exist as quantified acceptance criteria. Completion is defined by the gate. Reference exemplar (§4.4.1): AX, where a Stryker mutation gate drove MSI from 58.6% to 93.1% — proving <i>detection</i> , not merely line execution.

- **Automatable signal:** presence of blocking CI gates; mutation score (Stryker/mutmut) against threshold; coverage on changed lines; schema/contract checks present.
- **Human-judgment signal:** are tests written against interfaces or against the implementation? Would a behavior-preserving internal refactor wrongly fail, or a contract violation wrongly pass?

3.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
Test Theater (09)	0	Mutation testing as an adversarial audit; gate on MSI; run immediately after writing tests, not at release.
No Code Review Culture (12)	0	Behavioral contracts (API/contract verification, mutation testing) verify generated code automatically; the verification layer replaces manual correctness review.

4. Defended

4.1 Definition. Destructive operations are structurally prevented rather than merely discouraged. Commit hooks, branch protection, format enforcement, and MCP tool boundaries make classes of mistake architecturally unreachable; the system rejects malformed input the way a parser rejects a syntax error. A **gate is a production failure converted into structural prevention** — a field finding named as a forbidden pattern, encoded as a contract assertion, and promoted to a versioned gate carrying its originating incident as `provenance` (“each was paid for once”). The property extends to process (**RED-phase collapse** — no temporal barrier separates test authorship from implementation in a single context window; only structural gates, not instructions, close this) and, in zero-tolerance domains, to **consequence classification** (which operations are reversible / recoverable / irreversible, and the human confirmation gate before any irreversible action). **This is the one property CI cannot fully verify** (§4.4): whether adversarial challenge has been anticipated requires human review.

4.2 Lifecycle & organizational manifestation. - **Dev:** the agent physically cannot perform a forbidden action — a `feat:` without a prior `test:` `[RED]` is rejected; a destructive op without confirmation is blocked. - **Staging:** merge is blocked structurally on gate failure, not on a reviewer remembering to check. - **Production:** irreversible actions (`$C_i \gg o$`) require a named human confirmation gate; reversible-only domains absorb residual error via iteration. - **Evolution:** the gate corpus is cumulative and falsifiable — it grows only from observed failure; each gate is contestable by disputing its incident; gate-genesis proposes drafts from repeated friction. - **Toward the world:** the system cannot emit a class of defect that previously reached the world — the public surface is hardened by ledgered incidents. - **Toward the company:** compliance posture (PII redaction, audit logging, consequence tiers) is enforced, not aspirational; AI usage is governed (approved tools, MCP boundaries) rather than shadow. - **Toward the team:** discipline does not depend on vigilance or seniority — the gate enforces it uniformly across every contributor and every session, with no fatigue.

4.3 Scoring anchors.

Level	Anchor
0	Destructive operations are discouraged but not prevented: no commit hooks, no branch protection, unrestricted tool/shell access, no consequence classification, no PII/audit constraints where the domain requires them. TDD phase sequence is unenforced (RED-phase collapse). (Closest pathologies: <i>AI Security Blindspot</i> , <i>Shadow AI</i> , <i>Phase Collapse</i> , <i>Undisciplined Commits</i> .)
1	Some structural gates exist (e.g. conventional-commit lint, basic pre-commit, branch protection), but they are generic linter defaults rather than a provenance-bearing ledger; no consequence classification in a domain that needs it; adversarial challenge has not been systematically anticipated. Process gates (e.g. <code>[RED]</code> enforcement) are partial.
2	Destructive classes are architecturally unreachable: hooks/branch protection/tool boundaries block them; the gate corpus is cumulative and falsifiable, each entry carrying its originating incident as <code>provenance</code> ; TDD phase sequence is machine-enforced; in zero-tolerance domains, consequence tiers are classified with named human confirmation gates. A human reviewer confirms adversarial challenge has been answered. Reference exemplar (§4.4.1): AX, where Defended moved 0/2 → 2/2 only once gates were emitted as fenced file templates.

- **Automatable signal:** presence/firing of pre-commit and CI gates; branch-protection config; `[RED]` -before- `feat:` rule; PII-redaction and audit-logging gates where domain-required; each gate carries an incident reference.
- **Human-judgment signal (load-bearing here):** has adversarial challenge been anticipated and answered? Are the irreversible actions identified and gated? This judgment cannot be automated — the automated `gs_score` marks Defended provisional.

4.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
AI Security Blindspot (22)	0	Consequence classification (reversible/recoverable/irreversible); irreversible actions require human confirmation; PII redaction and audit logging as spec obligations.
Shadow AI (23)	0	Constitution governs all AI usage (approved tools, prohibitions); branch protection and MCP tool boundaries make mistakes unreachable.
Phase Collapse / TDD (10)	0–1	Structural phase separation: <code>[RED]</code> commit convention; pre-commit rejects test-only commits where all tests pass; CI blocks <code>feat:</code> without preceding <code>test:</code> .
Undisciplined Commits (17)	0–1	Conventional commits as required production rule; pre-commit hook running lint/typecheck/test; one logical change per commit.
Design Token Scatter (27)	1	Design tokens as a spec artifact; literal color/spacing values blocked at pre-commit (structural prevention at the visual layer).

5. Auditable

5.1 Definition. The current state of the system, and the history of how it arrived there, is fully recoverable from the artifacts alone. Conventional atomic commits form a typed corpus of change; ADRs document why the grammar evolved; status files record current implementation state. Nothing requires asking someone who was present. Without an auditable trail, the AI treats intentional architectural tradeoffs as defects to correct, producing drift silently. Full recoverability requires that commit discipline and the ADR record are *both* maintained; a spec without commit discipline provides only partial audibility — behavioral contracts survive, but the reasoning behind session-level decisions does not (the Shattered Stars boundary, §7.6).

5.2 Lifecycle & organizational manifestation. - **Dev:** the agent reads *why* a tradeoff was made before touching it, so it does not “optimize” a deliberate decision into a regression. - **Staging:** a reviewer reconstructs the rationale for a change from the ADR and commit corpus, not from a meeting. - **Production:** a defect is traceable to the introducing change via the typed commit log; current state is read from the status artifact. - **Evolution:** settled decisions are not re-litigated; an ADR closes a decision forever (superseded, never edited); debt is visible because deviation from spec is visible. - **Toward the world:** a change history is presentable — provenance for audits, certifications, due diligence. - **Toward the company:** bus factor collapses — knowledge is in artifacts, not people; M&A/audit red flags are removed because reasoning is recoverable. - **Toward the team:** vacations and departures do not strand the system; “only X knows why” stops being true.

5.3 Scoring anchors.

Level	Anchor
0	Neither decision history nor change history is recoverable: no ADRs, untyped/unreadable commits (“fix”, “WIP”), no status artifact. Rationale lives only in heads/Slack; the AI treats intentional tradeoffs as defects. (Closest pathologies: <i>ADR Absence</i> , <i>Undisciplined Commits</i> , <i>Bus Factor</i> , <i>Technical Debt Blindness</i> .)
1	Partial recoverability: either commit discipline <i>or</i> an ADR record exists, but not both maintained. Behavioral contracts and structural state survive across sessions, but the <i>provenance</i> of session-level decisions does not (the Shattered Stars boundary). Current state is partially captured.
2	Both maintained: conventional atomic commits form a typed, queryable corpus; ADRs (context/decision/consequences) record every non-obvious decision and are superseded rather than edited; a status artifact records current state. Why the system is the way it is, and how it got there, are recoverable from artifacts alone. Reference exemplar (§4.4.1): <code>pragmaworks docs/adrs/0001...0006</code> + conventional-commit history.

- **Automatable signal:** conventional-commit conformance over history; ADR directory present with structured entries; status artifact present and recently updated; commits traceable to changes.
- **Human-judgment signal:** pick a non-obvious decision in the code — can you recover *why* (alternatives considered, tradeoff accepted) from artifacts without asking anyone?

5.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
ADR Absence (16)	0–1	ADRs for every non-obvious decision (decision/context/consequences); store in <code>docs/adr/</code> ; never edit an accepted ADR — supersede it.
Undisciplined Commits (17)	0–1	Conventional commits as required production rule; the commit message describes a verified system state.
Bus Factor (14)	0	Externalize knowledge into the artifact set; the specification is the transfer mechanism, not the practitioner.
Technical Debt Blindness (15)	0–1	Auditable trail: deviation from spec = visible debt; ADRs and status make debt locatable and trackable.
Stale / Documentation Drift (19, 20)	1	Treat documentation as derivation from authoritative sources, keeping the audit trail truthful.
Unspecified Breakpoint Contracts (29)	1	ADR for responsive strategy and interaction patterns — records the decision so it is not re-made arbitrarily per session.

6. Composable

6.1 Definition. Units can be combined and extended without unexpected coupling. Clean architecture’s dependency inversion and the pure-function model ensure composition is predictable; the AI can work on any unit without unexpected propagation because isolation is structural, not assumed. Composable applies Dependency Inversion to the generation context: components must be navigable in isolation so a stateless reader can locate the relevant boundary without traversing the full artifact set. The structural isolation Composable requires also determines the safety of cross-cutting changes: AI search tools are not AST-aware (they match strings, not symbols), so a rename/interface change propagates to callers, re-exports, barrel files, and dynamic imports that `grep` cannot reliably find; in a system with clean Bounded boundaries, the change surface is exactly the boundary declaration. **Bounded and Composable together** close the search problem AST-less tooling leaves open.

6.2 Lifecycle & organizational manifestation. - **Dev:** the agent edits one unit and the change does not silently propagate; the relevant boundary is locatable without reading the whole graph. - **Staging:** a reviewer reasons about a unit in isolation; the impact set of an interface change is the boundary declaration, not “everywhere.” - **Production:** a fault in one unit does not cascade through unexpected coupling; isolation holds at runtime. - **Evolution:** cross-cutting changes (renames, interface changes) are safe because the change surface is declared, not discovered by grep; the system extends without re-coupling. - **Toward the world:** units compose into integrations predictably; an external component depends on a stable, declared interface. - **Toward the company:** features compose without integration surprises; the cost of combining units is predictable. - **Toward the team:** two engineers (or two AI sessions) extend two units in parallel without coupling collisions, because the seams enforce isolation.

6.3 Scoring anchors.

Level	Anchor
0	Coupling is implicit and unexpected: interfaces between components are unspecified, each side implements to its own assumptions, failures surface only at integration. Renames/interface changes propagate to unknown call sites; isolation is assumed, not structural. (Closest pathologies: <i>Implicit Contract Syndrome</i> , <i>Component Reinvention</i> .)
1	Boundaries exist (some dependency inversion, some interfaces), but isolation is partial: a unit cannot reliably be worked without checking neighbors; the change surface of an interface modification is larger than its boundary declaration; cross-language/cross-module contracts are not stated once in neutral terms.
2	Composition is predictable: dependency inversion at the composition root, interface-based seams, units navigable in isolation; the change surface for any interface modification is exactly the boundary declaration. (Requires Bounded to hold across seams.) Reference exemplar (§4.4.1): AX interface-based dependency injection — a stateless reader works a unit in isolation.

- **Automatable signal:** dependency-direction checks (e.g. depcruise: no inward-pointing violations, no route→DB); duplication metric (reinvented units inflate it); presence of explicit interface/contract artifacts; barrel/re-export surface bounded.
- **Human-judgment signal:** can a stateless reader work one unit without traversing the full set? Is the impact set of an interface change confined to its boundary?

6.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
Implicit Contract Syndrome (04)	0	Explicit API/inter-service contracts (signatures, inputs/outputs, error shapes, auth); every domain concept named once in neutral terms; both sides derive from the same vocabulary.
Component Reinvention (28)	0–1	Atomic-design library as the canonical, mandatory first source; compose from it instead of reinventing — preserves isolation and bounds duplication.

7. Executable

7.1 Definition. The generated output satisfies the behavioral contracts the specification defines **when exercised against a real execution environment** — not merely compiles and passes static analysis. Verifiable establishes that correctness checks *exist*; Executable establishes that the implementation *actually passes them against a live execution context*. A system can be fully Verifiable — correct types, passing lint, well-structured tests — while producing a server that fails every integration

test against a real database. Executable is **scored conditional on specification availability**: a formal contract (Hurl suite, OpenAPI diff, HL7 FHIR runner) enables automated measurement; a goal-directed program with only human acceptance criteria is scored **N/A**.

7.2 Lifecycle & organizational manifestation. - **Dev**: “done” requires the verify-and-correct loop to close against a live runtime, not just a green static check; ghost failures (infrastructure noise) are distinguished from real code bugs before fixing. - **Staging**: integration/contract suites run against a real (staging) environment, not mocks; behavioral contracts are exercised end to end. - **Production**: the deployed system meets its behavioral and non-functional contracts under real conditions (e.g. SLO ramp, p99 latency); contracts are validated, not assumed. - **Evolution**: the closed loop from production back to specification exists — a runtime failure becomes a spec query and a new gate; the system improves rather than drifting. - **Toward the world**: the live interface behaves as the published contract states for real consumers, under real load. - **Toward the company**: AI-accelerated code generation translates into shipped, deployable output — the downstream automation gap is closed, so velocity gains are realized, not bottlenecked. - **Toward the team**: engineers ship deployable implementations rather than code that compiles but does not run; comprehension is replaced by the verify loop, not by trust.

7.3 Scoring anchors.

Level	Anchor
0	The implementation does not pass its behavioral contracts against a live environment: it compiles and may pass unit tests, but integration/contract suites fail against a real database/runtime, or no live-environment execution is attempted. Infrastructure failures are mistaken for code bugs. (Closest pathologies: <i>Ghost Failure Cascade</i> , <i>Automation Gap</i> , <i>Copilot Dependency</i> .)
1	The implementation passes some behavioral contracts against a live environment but not the full suite — partial materialization (e.g. some integration/contract probes green, others failing); the verify-and-correct loop runs but is not closed; the loop from runtime back to spec is not yet structural.
2	The implementation satisfies its behavioral contracts against a real execution environment — formal contract suites (Hurl/OpenAPI/FHIR) pass against a live runtime, NFRs meet quantified thresholds, and the verify-and-correct loop is closed and gated. Reference exemplar (§4.4.1): EX <code>evidence/slo-ramp-summary.json</code> + the Hurl probe suite — contracts run against a live runtime.
N/A	No formal behavioral contract exists to run (goal-directed program, only human acceptance criteria). Record N/A; denominator drops to 12. Do not score 0 by default.

- **Automatable signal**: result of the contract suite against a live environment (Hurl/OpenAPI diff/FHIR runner); SLO/NFR ramp evidence (e.g. `slo-ramp-summary.json`); integration-verify exit status; ratio of suites that materialize.
- **Human-judgment signal**: is a failure a ghost (infrastructure) or a real code bug? For goal-directed programs, do human acceptance criteria pass against the running system?

7.4 Linked pathologies → remedies.

Pathology	Signal of low score	GS remedy that raises it
Ghost Failure Cascade (11)	0–1	Distinguish ghost failures from real bugs before building fix prompts; specify infrastructure state; include full test contents in fix prompts.
Automation Gap (25)	0–1	The obligation cascade removes downstream manual work (test/deploy/monitor/evolve) so code-gen gains reach shipped output — closing the loop to Executable.
Copilot Dependency (24)	0–1	Invert the relationship: spec + behavioral harness replace manual comprehension; output is verified executable, not trusted.

Cross-cutting note: properties below score and the system-level pathology

Some catalog pathologies are not the failure of a single property but the failure of the artifact set as a whole. The clearest is **Architectural Drift (01)** — locally valid, globally incoherent output that propagates silently across sessions. It is the aggregate symptom of low scores across Self-describing, Auditable, and Bounded together: no derivable intent, no recoverable rationale, no bounded context. A diagnostic that finds Architectural Drift should expect a low total `gs_score`, not a single low property. Similarly, **Annotation Fatigue (18)** is not a property gap but an executor problem — the human cost of maintaining a discipline exceeded its value; the GS remedy is to change the executor (the AI does not fatigue), then re-adopt the practice under structural enforcement.

Scoring refinements: declared rigor vs derivable guarantee

A recurring failure mode in spec-driven and phased-gate workflows is that the *shape* of rigor — reports, matrices, gates — is fully present while the *guarantee* is absent: status is authored, not derived. The following refinements make the rubric penalize declared-but-underived rigor. They sharpen **Verifiable**, **Defended**, and **Executable** in particular.

- **R1 — Evidence, not description.** No green status without an attached execution record (run id, inputs, outputs, timestamp). A report that *describes* a validation procedure which was never executed scores 0 on **Verifiable** and **Auditable** — prose about a test is not evidence of a test.
- **R2 — Computed status.** A “done” / “passing” status that is human-assigned rather than a function over evidence does not count. **Verifiable** forbids hand-typed status.
- **R3 — Enforced vs aspirational.** Every control declares whether it actually runs. **Defended** counts only the `enforced` set; a governance gate that exists as fitness tests but whose CI has never run is aspirational and scores 0 on that control.
- **R4 — Directionality test.** Score **Executable** by asking whether a stateless reader could regenerate the system from the specification alone. A spec whose requirements cite source files as their origin — reverse-engineered from the code — describes a system rather than specifying one, and fails Executable.
- **R5 — Intent-anchoring.** No capability is scorable until it traces upward to a job-to-be-done. “A stakeholder chose this technology” is not an intent; a module with no use case and no consumer fails **Self-describing** and **Bounded**.

- **R6 — Single source + embedded provenance.** One parameterized template rather than N near-duplicate files; content-addressed or embedded sources rather than external URLs with no snapshot; location-independent artifacts (no absolute paths). Violations penalize **Composable**, **Auditable**, and **Self-describing**.
- **R7 — Predicates over checklists.** A gate expressed as a human checklist (“all questions answered?”) does not score; a gate expressed as a machine-evaluable predicate (requirement→task coverage = 100%, no orphans) does. Keep the disciplinary kernel that moves a score; drop the human-gated document relay.

Negative test cases (specimens the rubric must score low — calibration set):

Specimen	Scores low on	Because
Validation report describing an unrun procedure	Verifiable, Auditable	prose ≠ evidence
Module with no use case / user story	Self-describing, Bounded	intent absent
Flagship output unimplemented under a “done” narrative	Verifiable, Defended	status not derived
Governance gate never wired to CI	Defended	control aspirational
Requirements citing source files as their origin	Executable	spec descriptive, not generative
Duplicated, post-hoc-amended artifacts	Composable, Auditable	no single source; drift

These derive from a field examination of a production-grade enterprise system built with a spec-driven phased-gate workflow; the positive patterns it also demonstrated — an executable policy layer, an ADR dependency graph, a per-decision constraint ledger, an honest gap register — are reflected in the property definitions above.

Harness-document YAML frontmatter schema (Self-describing, made machine-readable)

The Self-describing property requires that an artifact announce its own intent and scope. A short, **closed-key** YAML frontmatter block makes the rubric-relevant facts about a harness/spec document consumable by tooling — the sentinel router, the gates, and ForgeDX — **without parsing prose**. This is the Self-describing property rendered for a machine reader.

Apply the minimal-sufficient rule (harness-excess avoidance). Frontmatter is added *only where a tool consumes a key*. Over-marking — frontmatter on every file, free-form keys, decorative metadata no router or gate reads — is itself harness excess and re-creates the context-degradation the discipline prevents (cf. Bounded, §2). The keys are **closed**: a tool MAY reject an unknown key rather than silently tolerate drift. Free-form annotation belongs in prose, not frontmatter.

```

---
id: UC-014          # stable identifier; unique within the spec corpus
type: use-case      # closed enum: constitution | sentinel-node | spec-section | use-case
status: active      # closed enum: draft | active | superseded | archived
tier: T2            # obligation-cascade tier this artifact serves (T1..T5); see Compendiu
properties: [verifiable, executable] # which of the 7 this artifact serves; closed vocabulary
obligations: 4      # count of RFC-2119 normative clauses (MUST/SHOULD/MAY) the artifact c
generative_execution: green # closed enum: green | red | unrun – last live-environment result
depends_on: [UC-002, ADR-0006] # ids this artifact derives from or requires; powers the depend
---

```

Key contract (closed set):

Key	Meaning	Consumed by
<code>id</code>	Stable, unique identifier.	router (routing), ForgeDX (<code>evidence_signal</code> keying), <code>depends_on</code> resolution
<code>type</code>	Artifact kind (closed enum).	router (descent rules), gates (which gate applies)
<code>status</code>	Lifecycle state (closed enum).	router (skip <code>archived</code> / <code>superseded</code>), Auditable audit
<code>tier</code>	Obligation-cascade tier served (§4.3).	ForgeDX (<code>treatment_plan</code> sequencing), router
<code>properties</code>	Which of the 7 the artifact serves (closed vocabulary).	ForgeDX (<code>gs_score</code> attribution), gates (coverage check)
<code>obligations</code>	Count (or list) of RFC-2119 MUST/SHOULD/MAY clauses.	gates (normative-clause coverage), Verifiable
<code>generative_execution</code>	Last live-environment result: <code>green</code> <code>red</code> <code>unrun</code> .	gates (Executable signal), ForgeDX
<code>depends_on</code>	Upstream artifact ids.	router (load order), Composable (dependency-direction check), Auditable

Rules: keys are optional individually but **the set is closed** (no free-form keys); enum values are fixed; a document carries frontmatter **only if a listed consumer reads at least one of its keys**. A constitution root and a use case will carry frontmatter; a narrative essay should not.

Sentinel-tree nodes — the case where this pays off most. A node of the sentinel navigational tree (Bounded, Compendium §4.4) turns its prose scope-and-routing declaration into a machine-navigable, *verifiable* contract. A node carries `type: sentinel-node` plus four routing keys:

```

---
node: ecosystem      # node name within the tree
type: sentinel-node
scope: tools, repos, project map # one line: what this node covers
load: on-demand      # closed enum: always (root/core) | on-demand (descend when relevant)
categories: [routing, tool-sequencing] # which of the 5 required categories it contributes; clo
# architectural-identity | standards | constraints |
routes_to: [papers, files, website] # child node ids the agent may descend to
---

```

This makes the tree’s invariants checkable by a tool rather than asserted in prose: (1) the **root stays within the bounded line budget** and is the only node with `load: always` ; (2) across all nodes the

five required categories are **collectively present** — a gate can fail a tree that declares no `tool-sequencing` node, the most commonly absent and most consequential category (§4.4); (3) `routes_to` must resolve to real nodes, so the tree is provably connected and lossless. The router descends by `routes_to` deterministically instead of inferring from prose. ForgeCraft’s sentinel renderer (the source-of-truth that generates the per-agent copies) is the natural emitter and validator of this frontmatter, and its drift check verifies it; hand-written sentinels adopt it once that consumer is in place — the same minimal-sufficient rule.

How ForgeDX computes `gs_score` and maps it to treatment

ForgeDX consumes these anchors as its scoring contract. For each of the seven properties it extracts an `evidence_signal` from the assessed team’s documents and `survey` (the automatable signals above — gate presence, mutation score, ADR/commit conformance, line budgets, live-contract results, frontmatter keys), produces a diagnostic `hypothesis` per property, and selects the **0 / 1 / 2 anchor the evidence is closest to** (Executable may resolve to N/A, dropping the denominator to 12) — the same admissibility rule a human assessor follows. Summing the seven yields the `gs_score` (0–14). A property scoring 0 or 1 is the signal that a `pathology` from the linked-pathologies tables is present; ForgeDX names that `pathology`, attaches the corresponding `remedy` as a `prescription`, and orders the prescriptions by obligation-cascade `tier` into a phased `treatment_plan`. Because Defended cannot be fully machine-verified (§4.4), ForgeDX marks any Defended score derived without human review as **provisional** in the `assessment`, and flags it for human confirmation before the `treatment_plan` is finalized.